

1 - Rectangle

1. Définir une classe `Rectangle` ayant un attribut `longueur` et un attribut `largeur` de type `float` positifs.
2. Ajouter un constructeur pour instancier les attributs.
3. Ajouter les accesseurs / mutateurs (getters / setters) aux différents attributs sans utiliser les décorateurs `@property` ou `@x.setter`.
4. Ajouter les méthodes suivantes :
 - `perimetre(self)` : retourne le périmètre du rectangle.
 - `aire(self)` : retourne l'aire du rectangle.
 - `est_carre(self)` : vérifie si le rectangle est un carré.
 - `le_plus_grand(self, other)` : compare l'aire du rectangle « appellant » avec l'aire du rectangle « passé en paramètre » et retourne le plus grand des deux, si les deux sont égaux elle retourne l'un des deux de façon arbitraire.
 - `afficher(self)` : affiche les caractéristiques d'un rectangle comme suit :

```
Longueur : [...] - Largeur : [...] - Perimetre : [...] - Aire : ...[] - C'  
est un carre / Ce n'est pas un carre
```

5. Dans une fonction `main()`, créer plusieurs objets `Rectangle` et tester les méthodes.

2 - Poupées Russes

On veut écrire un programme simulant des poupées russes de différentes tailles. Chaque poupée a un nom, une taille donnée, peut s'ouvrir ou se fermer, peut contenir une autre poupée et être contenue dans une autre poupée.

1. Définir une classe `PoupeeRusse` ayant les attributs privés suivants :
 - `nom` (chaîne)
 - `taille` (entier positif)
 - `est_ouverte` (booléen)
 - `dans` (contenue dans une autre poupée)
 - `contient` (contient une autre poupée).
2. Implémenter des accesseurs (getters) pour tous les attributs avec `@property`.
3. Après avoir lu tout l'énoncé de l'exercice, implémenter les mutateurs (setters) qui vous semblent pertinents avec `@x.setter`.
4. Implémenter un constructeur initialisant correctement tous les attributs. On considérera que toutes les poupées sont fermées et vides à leur création.

5. Ajouter les méthodes suivantes :

- `ouvrir(self)` : ouvre la poupée si elle n'est pas déjà ouverte et si elle ne se trouve pas à l'intérieur d'une autre poupée.
- `fermer(self)` : ferme la poupée si elle n'est pas déjà fermée et si elle ne se trouve pas à l'intérieur d'une autre poupée.
- `placer_dans(self, p)` : place la poupée courante dans la poupée `p` si elle n'est pas déjà dans une autre et si la poupée `p` ne contient aucune autre poupée et si la poupée courante est fermée et la poupée `p` est ouverte et plus grande.
- `sortir_de(self)` : sort la poupée courante d'une autre poupée si la contenante est bien ouverte.
- `__str__(self)` : qui retourne les informations de la poupée sous forme de chaîne. Attention à bien retourner le nom des poupées contenues / contenant.

6. Dans une fonction `main()`, créer plusieurs objets `PoupeeRusse` et tester les méthodes.

3 - Pokémons

1. Définir une classe `Pokemon` qui contient les attributs suivants :

- Un attribut `nom` qui représente le nom du Pokémon avec une chaîne ;
- Un attribut `pv` qui représente les points de vie du Pokémon avec un entier positif ;
- Un attribut `atk` qui représente les points de dégâts du Pokémon avec un entier positif.

2. Implémenter tous les getters ainsi que les setters pertinents puis un constructeur.

3. Implémenter une propriété `est_ko(self)` qui retourne un booléen indiquant s'il ne reste plus de PV au Pokémon.

4. Implémenter les méthodes suivantes :

- `attaquer(self, autre)` : permet au Pokémon `self` d'attaquer `autre` en lui retirant un certain nombre de points de vie. Ce nombre est tiré aléatoirement entre 0 et `atk`.
- `combattre(self, autre)` : permet aux deux Pokémon de s'attaquer à tour de rôle. La méthode retourne le gagnant et le nombre de tours d'attaque.
- `__str__(self)` : retourne les informations du Pokémon sous forme de chaîne.

5. Définir les classes `PokemonNormal`, `PokemonFeu`, `PokemonEau` et `PokemonPlante` qui héritent de la classe `Pokemon`. Ces classes filles implémentent que certains types font plus ou moins de dégâts à d'autres types :

- Les Pokémon feu font deux fois plus de dégâts aux Pokémon plante mais font deux fois moins de dégâts aux Pokémon eau ou feu.

- Les Pokémons eau font deux fois plus de dégâts aux Pokémons feu mais font deux fois moins de dégâts aux Pokémons eau ou plante.
- Les Pokémons plante font deux fois plus de dégâts aux Pokémons eau mais font deux fois moins de dégâts aux Pokémons plante ou feu.
- Les Pokémons normaux infligent et reçoivent des dégâts normaux de tous les types.

On pourra s'aider d'une fonction `calc_multiplieur(self, autre)` surchargée dans les classes filles afin de retourner 0.5, 1 ou 2 en fonction du type de `autre`.

6. Dans une fonction `main()`, créer des `Pokemon` de différents types et amusez-vous à faire des combats de Pokémon.